

Software Requirements Specification (SRS)

UrbanHeat Accra — Machine-Learning-Based Urban Heat Risk Prediction & Mitigation-Simulation Dashboard

Document Version: 1.0

Date: 15 August 2026

Student Name: Emmanuel Owusu

Student ID: 22425075

Table of Contents

1. Introduction
 2. Purpose
 3. Scope
 4. Product Perspective
 5. Stakeholders
 6. User Classes
 7. Functional Requirements
 8. Non-Functional Requirements
 9. System Interfaces
 10. Database Requirements
 11. Authentication Requirements
 12. Security Requirements
 13. Constraints
 14. Assumptions
 15. Acceptance Criteria
 16. Requirements Prioritisation
 17. Requirements Traceability
-

1. Introduction

This Software Requirements Specification (SRS) defines the complete set of functional and non-functional requirements for the UrbanHeat Accra system. It is intended to serve as the definitive reference for the system's expected behaviour, constraints, and acceptance criteria.

1.1 Document Conventions

- **Must, Should, Could, Won't:** MoSCoW priority indicators
- **FR:** Functional Requirement
- **NFR:** Non-Functional Requirement
- **UC:** Use Case

1.2 Intended Audience

- Academic supervisor/examiner
- Developer (student)
- Future maintainers of the system

1.3 References

- Project_Documentation.pdf — Master project report
 - Testing_Report.pdf — Standalone testing documentation
 - Technical_Debt_Plan.pdf — Technical debt register and roadmap
-

2. Purpose

The purpose of the UrbanHeat Accra system is to provide urban planners and environmental officers in the Greater Accra Metropolitan Area with a web-based decision support tool that:

1. Predicts heat risk at individual location level using machine learning
 2. Explains the environmental factors contributing to each location's risk level
 3. Enables simulation of vegetation-based mitigation interventions with quantitative before/after comparison
 4. Presents all information in a visually intuitive, non-technical interface
-

3. Scope

3.1 In Scope

- Interactive map dashboard showing 250 monitored locations across Accra
- Location-level heat risk scores (0–100) with risk categories (Low, Moderate, High, Severe)
- Feature importance explanation using Random Forest feature_importances_
- Vegetation increase simulation with before/after risk comparison
- Data explorer with sortable, searchable, exportable location table
- Custom risk prediction from user-defined environmental parameters
- RESTful API with automated Swagger documentation

3.2 Out of Scope

- Real-time satellite data ingestion
 - Multi-city support
 - User authentication and session management
 - Administrative data management (CRUD operations on locations)
 - Mobile native application
 - Integration with external municipal IT or GIS systems
 - Real-time temperature sensor data
-

4. Product Perspective

UrbanHeat Accra is a standalone, self-contained web application. It does not integrate with or depend upon existing municipal information systems. The application consists of:

- A **React single-page application (SPA)** served from Vercel
- A **FastAPI REST backend** hosted on Render
- A **SQLite database** storing 250 pre-seeded location records
- A **pre-trained scikit-learn RandomForest model** loaded at backend startup

The frontend communicates with the backend exclusively through RESTful JSON API calls over HTTPS.

5. Stakeholders

Stakeholder	Role	Interest Level	Influence Level
Urban Planners	Primary end users — identify and prioritise heat-vulnerable areas	High	High
Environmental Officers	Monitor environmental risk indicators	High	Medium
City Administration	Decision-makers for resource allocation	Medium	High
Residents of High-Risk Areas	Indirect beneficiaries of planning decisions	High	Low
Academic Supervisor	Evaluate technical quality and documentation	High	High
Developer (Student)	Design, build, test, and document the system	High	High

6. User Classes

6.1 Urban Planner (Primary User)

- **Technical Proficiency:** Low to moderate; no ML background assumed
- **Access Pattern:** Desktop browser, occasional mobile
- **Primary Tasks:** View heat risk map, inspect locations, run simulations, export data
- **Key Need:** Plain-language explanations, before/after comparisons, spatial context

6.2 Environmental Officer

- **Technical Proficiency:** Moderate
- **Access Pattern:** Desktop browser
- **Primary Tasks:** Review risk data, export datasets for reports
- **Key Need:** Tabular data access, CSV export, sortable/filterable views

6.3 System Administrator (Future — Not in MVP)

- **Technical Proficiency:** High
- **Primary Tasks:** Manage location data, update model, configure system
- **Status:** Out of scope for MVP (FR9 Won't)

7. Functional Requirements

FR1: Interactive Heat Risk Map

Priority: Must

The system shall display all monitored locations on an interactive web map, with each location represented by a colour-coded pin indicating its heat risk category:

- Green: Low risk (score < 30)
- Amber: Moderate risk (score 30–55)
- Red/Terracotta: High risk (score 55–75)
- Dark Red: Severe risk (score ≥ 75)

The map shall use OpenStreetMap/CARTO tile layers and centre on Accra, Ghana (approximately 5.603°N, 0.187°W).

FR2: Location Selection and Detail View

Priority: Must

The system shall allow users to select a location by:

1. Clicking a map pin, OR
2. Typing a location name or neighbourhood into a search box with autocomplete suggestions

Upon selection:

- The map shall animate (fly) to the selected location
- A detail drawer shall slide in from the right showing: location name, coordinates, heat risk score (0–100), risk category badge, neighbourhood, and environmental baseline values (NDVI, built-up density, green space distance, elevation)

FR3: Contributing Factor Explanation

Priority: Must

For each selected location, the system shall display the top contributing environmental factors for its risk score, presented as:

- A horizontal bar chart ranked by feature importance
- Colour-coded bars (red for risk-increasing, green for risk-decreasing)
- Percentage importance values
- A plain-language insight sentence summarising the top factor

The data shall be sourced from the GET /api/explain/{id} endpoint.

FR4: Vegetation Mitigation Simulation

Priority: Must

The system shall allow users to simulate a vegetation cover increase on any selected location:

- Preset buttons: + 10%, + 20%, + 30%, + 50% vegetation
- Custom slider: 0–100% vegetation increase
- Display: before/after risk score comparison, delta badge, NDVI shift bar
- Disclaimer: "Simulated using a simplified vegetation model — indicative only"
- Reset: "Reset to Baseline" button to clear simulation

The data shall be sourced from the POST /api/simulate endpoint.

FR5: RESTful API

Priority: Must

The backend shall expose a RESTful API with the following validated endpoints:

Endpoint	Method	Input	Output
/api/health	GET	—	{"status": "ok"}
/api/locations	GET	Optional: min_risk, max_risk, sort_by	Location list with count
/api/locations/{id}	GET	Path: location_id	Single location
/api/explain/{id}	GET	Path: location_id	Risk score + top_factors array
/api/predict	POST	ndvi, built_up_density_pct, distance_to_green_space_m, elevation_m	risk_score, risk_category
/api/simulate	POST	location_id, delta_vegetation_pct	before/after scores and categories

All endpoints shall return JSON. POST bodies shall be validated with appropriate error responses (422) for invalid input.

FR6: Data Explorer

Priority: Should

The system shall provide a data explorer view with:

- A table showing all location records with columns: name, neighbourhood, coordinates, NDVI, built-up %, green space distance, elevation, risk score, risk category
- Sortable columns (click header to sort ascending/descending)
- Text search filtering by name or neighbourhood
- Pagination (12 records per page)
- CSV export of filtered/sorted data
- Click-to-locate button to jump to location on map

FR7: Risk Category Filter

Priority: Should

The system shall allow users to filter map pins and data by risk category using segmented control buttons:

- All Risks (default)

- Low (< 30)
- Moderate (30–55)
- High (55–75)
- Severe (> 75)

Filtered count shall update in KPI cards.

FR8: Custom Risk Prediction Tool

Priority: Should

The system shall provide a modal-based prediction tool where users can:

- Adjust environmental parameters via sliders: NDVI (-0.2 to 0.9), built-up density (0–100%), green space distance (0–5000m), elevation (0–120m)
- Submit parameters to `/api/predict`
- View the predicted risk score and category

FR9: Administrative Data Management

Priority: Won't (Descoped)

The system shall NOT provide an admin interface for adding, editing, or deleting location data in this MVP version.

8. Non-Functional Requirements

NFR1: Performance

Prediction API responses (`/api/predict`) shall complete within 1 second under normal operating conditions. Achieved by pre-loading the ML model at application startup.

NFR2: Usability

The interface shall be usable by urban planners with no machine learning or technical background. All labels shall use domain-specific plain language (e.g., "High Risk" not "Class 2"; "Vegetation Index" not "NDVI model coefficient").

NFR3: Browser Compatibility

The frontend shall function correctly on the latest versions of Google Chrome, Mozilla Firefox, and Microsoft Edge.

NFR4: Responsiveness

The frontend shall be responsive across desktop ($\geq 1024\text{px}$), tablet (600–1023px), and mobile ($< 600\text{px}$) viewports. Primary optimisation is for desktop; functional on mobile.

NFR5: Portability

The system shall be deployable without external database services by default (SQLite). The

database shall be upgradable to PostgreSQL by changing only the DATABASE_URL environment variable.

NFR6: Security / Input Validation

- All POST endpoint request bodies shall be validated server-side with explicit range constraints
- Invalid input shall return HTTP 422 with field-level error descriptions
- Internal server errors shall never expose stack traces (generic error message returned)
- CORS shall be restricted to the configured frontend origin

NFR7: Explainability

Every displayed heat risk score shall be accompanied by explainability information — specifically, the top contributing environmental factors ranked by importance with directional indication (increases or decreases risk).

NFR8: Reliability / Monitoring

The API shall provide a health check endpoint (GET /api/health) returning `{"status": "ok"}` for deployment monitoring and automated health checks.

9. System Interfaces

9.1 Frontend ↔ Backend Interface

- **Protocol:** HTTPS
- **Format:** JSON
- **Authentication:** None (public API)
- **Base URL (Production):** `https://urban-heat-backend.onrender.com`
- **Base URL (Development):** `http://127.0.0.1:8000` (proxied by Vite dev server)

9.2 Backend ↔ Database Interface

- **ORM:** SQLAlchemy 2.0 +
- **Driver:** `sqlite3` (built-in) or `psycopg2` (PostgreSQL)
- **Connection:** Managed via `SessionLocal` + dependency injection

9.3 Backend ↔ ML Model Interface

- **Loading:** `joblib.load()` at module import time
- **Model File:** `heat_risk_model.pkl` (bundled in `app/` directory)
- **Inference:** `model.predict(DataFrame) → float (risk_score)`
- **Explainability:** `model.feature_importances_ → float array`

9.4 External Interfaces

- **Map Tiles:** CARTO Voyager basemap tiles (via Leaflet `TileLayer`)
- **Hosting:** Render (backend), Vercel (frontend)

10. Database Requirements

10.1 Schema

Single table `locations` with 10 columns (see Section 7, FR5 for schema detail).

10.2 Data Volume

250 location records, ~40 KB total database size.

10.3 Data Integrity

- Primary key constraint on `id`
- NOT NULL constraints on all columns except `neighbourhood`
- Index on `id` (primary key) and `name`

10.4 Data Seeding

Database is populated from `seed_locations.csv` on first startup. Idempotent — skips if data already exists.

10.5 Upgrade Path

SQLite → PostgreSQL by changing `DATABASE_URL` environment variable. No code changes required.

11. Authentication Requirements

11.1 Current Version (MVP)

No authentication is implemented. All endpoints are publicly accessible. This is a deliberate design decision:

- The system serves non-sensitive, publicly available environmental data
- The MVP is read-only (no data mutation endpoints)
- FR9 (admin panel) was descoped under MoSCoW prioritisation

11.2 Future Version

JWT-based authentication with role-based access control is planned for Version 2.0 (see [Technical_Debt_Plan.pdf](#), TD-03).

12. Security Requirements

SR1: CORS

The backend shall restrict cross-origin requests to the configured `FRONTEND_ORIGIN`

environment variable. Only GET, POST, and OPTIONS methods shall be allowed.

SR2: Input Validation

All user-submitted data shall be validated server-side using Pydantic schemas with explicit constraints. Invalid input shall return HTTP 422.

SR3: Error Masking

Internal server errors (500) shall not expose stack traces, implementation details, or sensitive information to the client.

SR4: Environment Secrets

Database URLs, API keys, and other sensitive configuration shall be stored in environment variables (.env), excluded from version control via .gitignore.

SR5: HTTPS

All production communication shall use HTTPS, enforced by the hosting platforms (Render, Vercel).

SR6: No Data Mutation

No API endpoints shall allow creating, updating, or deleting location data in the current MVP version.

13. Constraints

Constraint	Impact
48-hour development time limit	Limits feature scope, test coverage, and mobile optimisation
Solo developer	No peer review; all design decisions made individually
Synthetic training data	Model predictions are illustrative, not calibrated
Free-tier hosting (Render, Vercel)	Potential cold-start latency on Render free tier
No access to real satellite imagery API	Cannot validate model against real-world data
Academic submission deadline	Limits scope of Technical Debt repayment

14. Assumptions

1. Users have access to a modern web browser (Chrome, Firefox, or Edge) with JavaScript enabled.
2. Users have a stable internet connection to access the hosted application.
3. The 250 seed locations are representative of the Greater Accra Metropolitan Area.
4. The synthetic feature-to-risk relationships in the training data are physically plausible (higher NDVI → lower risk, higher built-up density → higher risk, etc.).

5. The Random Forest model generalises adequately within the feature ranges present in the dataset.
 6. Render and Vercel hosting platforms remain available and operational.
 7. No concurrent database writes are needed (SQLite is sufficient for read-heavy MVP).
-

15. Acceptance Criteria

ID	Acceptance Criterion	Verification Method
AC1	Map displays all 250 locations with correct colour coding	Visual inspection
AC2	Clicking a pin opens the detail drawer with correct data	Functional test
AC3	/api/predict returns a valid risk score (0–100) for valid input	TC3
AC4	/api/predict returns 422 for out-of-range NDVI	TC4
AC5	/api/simulate shows after_risk ≤ before_risk for positive vegetation delta	TC7
AC6	/api/explain returns non-empty top_factors array	TC6
AC7	Data Explorer displays all locations with sorting and search	Visual inspection
AC8	CSV export produces a valid CSV file	Functional test
AC9	Health endpoint returns {"status": "ok"}	TC1
AC10	System loads within 5 seconds on a standard broadband connection	Performance check
AC11	No stack traces are exposed in any error response	TC5, TC4, TC8
AC12	System deploys successfully to Render and Vercel	Deployment verification

16. Requirements Prioritisation

Priority	Requirements
Must	FR1, FR2, FR3, FR4, FR5, NFR1, NFR5, NFR6, NFR7, NFR8
Should	FR6, FR7, FR8, NFR2, NFR3, NFR4
Could	Custom vegetation slider, per-instance SHAP, data pagination
Won't	FR9, real-time satellite data, multi-city, authentication

17. Requirements Traceability

Requirement	Use Case	Component	API Endpoint	Test Case
FR1	UC1	MapView.jsx	GET /api/locations	TC2
FR2	UC2	Header.jsx, MapView.jsx, LocationDrawer.jsx	GET /api/locations/{id}	TC2
FR3	UC3	LocationDrawer.jsx (Overview tab)	GET /api/explain/{id}	TC6
FR4	UC4	LocationDrawer.jsx (Simulate tab)	POST /api/simulate	TC7

FR5	—	main.py, schemas.py	All endpoints	TC1–TC8
FR6	UC5	DataExplorerer.jsx	GET /api/locations	Visual
FR7	UC1	Header.jsx	GET /api/locations	Visual
FR8	UC6	PredictModal.jsx	POST /api/predict	TC3, TC4
NFR1	—	ml.py	POST /api/predict	TC3 timing
NFR6	—	schemas.py, main.py	All POST	TC4, TC8
NFR7	UC3	ml.py, LocationDrawer.jsx	GET /api/explain	TC6
NFR8	—	main.py	GET /api/health	TC1

End of Software Requirements Specification